

Agentic AI Course - Day 4

Krish Naik - 10h course | Video 2:00-2:40

LangGraph Basics - State, Nodes, Edges

LangGraph models an app as a graph. You define a shared State, add Nodes (functions that update state) and Edges (what runs next). This makes multi-step, controllable agents.

Key concepts

- State: a typed dict passed between nodes; each node returns updates to it.
- Node: a function (state) -> partial state. It does one unit of work.
- Edge: connects nodes; START -> node -> END defines the flow.
- Compile the graph, then .invoke(initial_state) to run it.

Code example

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict

class S(TypedDict):
    text: str

def upper(s): return {'text': s['text'].upper()}

g = StateGraph(S)
g.add_node('upper', upper)
g.add_edge(START, 'upper'); g.add_edge('upper', END)
app = g.compile()
print(app.invoke({'text': 'hi'}))
```

Today's tasks (1.5h)

- [] Define a TypedDict State
- [] Add one node and START/END edges
- [] Compile and invoke the graph

Resources

Video: youtu.be/rV3HJ4LEZ7k (start at 2:00-2:40)

Code: github.com/krishnaik06/Agentic-LanggraphCrash-course

(Full clickable links are in the app: Goals > this day > Study block.)